

Assignment3_b_code_student.ipynb

```
In [1]: # ELEC6604 Assignment 3B demo python file
```

```
In [9]: # PART 1:
# CONNECT WITH YOUR DRIVE IF IN COLAB

mode = 'jupyter' # mode = 'colab' or 'jupyter'

if mode == 'colab':
    from google.colab import drive
    drive.mount('/content/drive')
```

```
In [15]: # LOAD IN THE LIBRARIES
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical

if mode == 'colab':
    # for Colab: keras imports for the dataset and building our neural net
    from keras.datasets import cifar10
    from keras.models import Sequential
    from keras.layers import Dense, Dropout, Conv2D, MaxPool2D, Flatten
    from keras.utils import np_utils

# for Jupyter notebook, you have to import the necessary library
# .....
```

```
# PART 2A: THE FOLLOWING IS ONLY FOR jupyter notebook RUNNING  
# Loading the dataset directly from Internet  
if mode == 'jupyter' :  
    from tensorflow.keras.datasets import cifar10  
    (X_train, y_train), (X_test, y_test) = cifar10.load_data()  
  
    print('X_train.shape is ', X_train.shape)  
    print('y_train.shape is ', y_train.shape)  
    print('X_test.shape is ', X_test.shape)  
    print('y_test.shape is ', y_test.shape)
```

```
X_train.shape is (50000, 32, 32, 3)  
y_train.shape is (50000, 1)  
X_test.shape is (10000, 32, 32, 3)  
y_test.shape is (10000, 1)
```

```
# PART 3: THE FOLLOWING IS THE SAME CODING FOR BOTH COLAB and JUPYTER NOTE  
#
```

```
X_train = X_train.astype('float32')  
X_test = X_test.astype('float32')
```

```
# normalizing the data to help with the training  
X_train /= 255  
X_test /= 255
```

```
# PART 4: Investigation on Normalization
```

```
# What is the significance of the following step in this section ?  
# Examine whether the step has helped in the training process.  
# Basically, you would like to examine whether the training result is affected  
# with and without running the following two lines.
```

```
X_train = (X_train - 0.5)*2
```

```
X_test = (X_test - 0.5)*2
```

PART 5: one-hot encoding using keras' numpy-related utilities

```
n_classes = 10
```

```
print("Shape before one-hot encoding: ", y_train.shape)
```

```
# Y_train = np_utils.to_categorical(y_train, n_classes)
```

```
# Y_test = np_utils.to_categorical(y_test, n_classes)
```

```
Y_train = to_categorical(y_train, n_classes)
```

```
Y_test = to_categorical(y_test, n_classes)
```

```
print("Shape after one-hot encoding: ", Y_train.shape)
```

Shape before one-hot encoding: (50000, 1)

Shape after one-hot encoding: (50000, 10)

Double-check on the dimension

```
print('X_train.shape is ', X_train.shape)
```

```
print('Y_train.shape is ', Y_train.shape)
```

```
print('X_test.shape is ', X_test.shape)
```

```
print('Y_test.shape is ', Y_test.shape)
```

X_train.shape is (50000, 32, 32, 3)

Y_train.shape is (50000, 10)

X_test.shape is (10000, 32, 32, 3)

Y_test.shape is (10000, 10)

```
# PART 6 : CNN model development
# .....
# .....
```

```
# JUPYTER NOTEBOOK CODING
# Plots for training and testing process: loss and accuracy
```

```
plt.figure(0)
plt.plot(cnn.history['acc'], 'r')
plt.plot(cnn.history['val_acc'], 'g')
plt.xticks(np.arange(0, 101, 2.0))
plt.rcParams['figure.figsize'] = (8, 6)
plt.xlabel("Num of Epochs")
plt.ylabel("Accuracy")
plt.title("Training Accuracy vs Validation Accuracy")
plt.legend(['train', 'validation'])
```

```
plt.figure(1)
plt.plot(cnn.history['loss'], 'r')
plt.plot(cnn.history['val_loss'], 'g')
plt.xticks(np.arange(0, 101, 2.0))
plt.rcParams['figure.figsize'] = (8, 6)
plt.xlabel("Num of Epochs")
plt.ylabel("Loss")
plt.title("Training Loss vs Validation Loss")
plt.legend(['train', 'validation'])
```

```
plt.show()
```